

Feature-Trace: Generating Operational Profile and Supporting Testing Prioritization from BDD Features

Rafael Fazzolino

fazzolino29@gmail.com

University of Brasilia

Department of Computer Science

Genáina Nunes Rodrigues

genaina@unb.br

University of Brasilia

Department of Computer Science

ABSTRACT

Operational Profiles provide quantitative information about how the software will be used, which supports highlighting those software components more sensitive to reliability based on their profile usage. However, the generation of Operational Profiles usually requires a considerable team effort to liaise requirements specification until their reification into expected software artifacts. In this sense, it becomes paramount in the software life cycle the ability to seamlessly or efficiently perform traceability from requirement to code, embracing the testing process as a means to ensure that the requirements are satisfiably covered and addressed. In this work, we propose the Feature-Trace approach which merges the advantages of the Operational Profile and the benefits of the requirements-to-code traceability present in the BDD (Behavior-Driven Development) approach. The primary goal of our work is to use the BDD approach as an information source for the semi-automated generation of the Operational Profile. The proposed approach was evaluated on the Diaspora software, on a GitHub open source software. The case study revealed that the Feature-Trace approach is capable of extracting the operational profile seamlessly from the specified Diaspora's BDD features as well as obtaining and presenting vital information to guide the process of test cases prioritization.

CCS CONCEPTS

• **Software and its engineering** → **Process validation; Traceability; Software development techniques.**

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SBES 2019, September 23–27, 2019, Salvador, Brazil

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-7651-8/19/09...\$15.00

<https://doi.org/10.1145/3350768.3350781>

KEYWORDS

operational profile, behavior-driven development, requirements traceability, testing

ACM Reference Format:

Rafael Fazzolino and Genáina Nunes Rodrigues. 2019. Feature-Trace: Generating Operational Profile and Supporting Testing Prioritization from BDD Features. In *XXXIII Brazilian Symposium on Software Engineering (SBES 2019), September 23–27, 2019, Salvador, Brazil*. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3350768.3350781>

1 INTRODUCTION

Given the notorious challenge regarding the verification of software completeness and complexity through test cases [5], it is paramount to delimit a relevant set of test cases that guarantees the highest possible reliability within the limitations of the software development team [21]. Aiming at this goal, several techniques for prioritizing test cases have been proposed, related to the prioritization of either white-box or black-box testing or both [14]. These strategies use the data from the system under test (SUT) to define the prioritization criteria of test cases, such as the complexity of code units (effort to test that unit) [8], Operational Profile (OP) information [19], history of failures [23] and similarity of the test cases [18], among others [5].

In particular, many techniques are based on concepts such as OP, which takes into account the probability of execution of each entity of the software to provide quantitative information about how the software will be used [20]. Although the concept of OP allows a very interesting prioritization process, there are several limitations that often render impractical the use of the OP in the software development cycle, as the effort and the complexity involved in generating the OP.

Moreover, the success of software testing activity is directly connected to the correct alignment (traceability) between requirements and test cases [4]. Also, communication between requirements engineers and test engineers is often compromised by dealing with abstraction levels, languages and entirely unconnected complexities [26]. Thus, several studies have proposed strategies and techniques for locating requirements in the source code [1, 7, 9, 15, 16]. Among the

approaches used with this objective and which are gaining notoriety is the *Behavior Driven Development* (BDD) [24].

Given the gaps and difficulties presented to ensure the traceability between requirements and software testing, as well as the lack of an approach that provides a solid way of applying the concepts inherent to the OP, we propose the *Feature-Trace* approach. Feature-Trace facilitates the understanding of the software product from the perspective of the tested requirements as BDD artifacts and their attributed relevance. Then, leveraged by the construction of the OP from the BDD artifacts, Feature-Trace enables the traceability, testability and quantitative measure of importance from the requirements to the code units and their corresponding test cases.

The proposed approach was evaluated on the Diaspora social network software, an open source project, which contains 68 BDD features, specified in 267 scenarios. The case study revealed that the Feature-Trace approach is capable of extracting the OP as well as obtaining and presenting vital information to guide the prioritization process of test cases.

The remaining sections are organized as follows: Section 2 presents key background concepts. Section 3 highlights our Feature-Trace approach. Section 4 presents the case study carried out on the Diaspora software. Section 5 provides an overview on related work. Finally Section 6 concludes the paper and outlines future works.

2 BACKGROUND

Operational Profile

The Operational Profile aims to quantify the probability of occurrence of each entity of the software, making it possible to obtain the importance of each entity when it refers to the impact on the reliability of the software [20].

An operational profile consists mainly of identifying the entities at each level of the OP and obtaining their respective probabilities of occurrence. According to [19], one must first identify the top-level entities, beginning with *Customer Profile*, which represents a cluster of *Users Profiles* that similarly use the software. Each *User Profile* represents how a set of users with a similar profile use the software. The third level, *System-mode Profile*, represents modes of use of the software, such as *web* and *mobile* environments. The next level, *Functional Profile* represents how each feature will be used in the production environment, i.e., how likely each feature of the software will occur. The features have some variations, based on the input used to perform the functionality. These variations are represented in the *Operational Profile*, by the concept of *Key-input Variable*. In the last level, the *Operational Profile* represents the probability of occurrence of each operation that makes up certain functionality.

Behavior Driven Development

According to [28], the BDD approach enables the development team to focus their efforts on the identification, understanding, and construction of features of business importance, allowing these resources to be well designed and well implemented. The BDD approach is a derivation of the concepts of TDD (Test Driven Development) [2] and DDD (Domain Driven Design) [11], including an easy-to-understand language so that all stakeholders can understand and collaborate with writing requirements.

Among the ubiquitous language formats used by the BDD approach, we highlight *Gherkin*¹, which has a set of syntactic rules that allow the writing of scenarios of use such that, with the support of the appropriate tool, they can be mapped to source code methods and executed automatically to validate their behavior. The BDD approach consists of a “feature”, which has one or many “scenarios”, where each “scenario” has one or many “steps”. Using this approach, feature-to-code traceability can be obtained automatically, since the system requirements become an executable and verifiable entity.

3 THE FEATURE-TRACE APPROACH

Our approach, *Feature-trace*, aims to define a method for the semi-automated generation of the OP and to allow for the prioritization of software testing by means of BDD artifacts and quantitative SUT information. Our approach uses BDD artifacts to enable the generation of the OP, while minimizing the usual effort involved in the OP synthesis process. As such, *Feature-Trace* allows for the seamless analysis of the distribution of the test effort employed throughout the source code and the prioritization and selection of test cases based on OP, program spectrum, number of impacted features, and the methods complexity.

The Feature-Trace approach can be organized into three main modules: (1) *Automated Runtime Module* (ARM), (2) *Analyzer* and (3) *Visualizer*. These will be presented below.

The Automated Runtime Module

The main objective of the ARM is to extract information from the SUT, statically and dynamically, persisting the information required to carry out the analysis present in the other two modules: *Analyser* and *Visualizer*. The main activity of the ARM is to extract the entities needed to generate the OP, both from the BDD artifacts and the source code.

Regarding the OP entities extraction, we start from the highest level of the OP. We should note that the BDD approach does not directly support the definition of an entity related to it. However, BDD allows the inclusion of user-defined “tags” to group features into similar groups. This

¹<https://cucumber.io/docs/gherkin/>

capability can be used to define the **Customer Profile**, the **User Profile** and **System-mode Profile** level. The **Functional Profile** level can be related directly to the concept of BDD “Feature”, which also represents the functionality of the software in the view of the user [24].

After identifying the **functions**, the **key input variables**, which define the variations of the use of each **function**, must be raised [19]. It can be obtained directly from the concept of BDD scenarios. Finally, at the last level is the **Operational Profile**, which is obtained from the execution trace analysis of each **key input variable**. That is, operations are mapped into entities of the SUT source code enabling the execution of specific functionality. In this work, operations are mapped in software methods, but the degree of granularity must be adapted according to the OP analysis need.

Basically, the proposed relationship between the entities of the OP and the entities of the BDD approach are: (1) *Customer*, *User* and *System mode* are related by *User-defined tags*, (2) *Function* related by BDD *feature*, (3) *Key-input Variable* related by BDD *Scenario*, and (4) *Operational Profile* related by *Scenarios trace*.

Following these relationships, the ARM uses automation strategies to obtain all the entities of the OP, including all execution traces. In addition to this information, we obtain, statically, information about the complexity of each method, such as *ABC score* [12], and *Cyclomatic Complexity* [17]. In addition, the ARM obtains the result of each test case, recording whether the test has passed or failed.

The Analyser Module

This module is responsible for constructing the mapping between the entities of the software, relating from the requirements to the computational units that compose them. The output of the information provided by the ARM is incrementally submitted to the Analyzer Module, where each feature is executed individually. By these means, the Analyzer Module receives requests sequences containing a feature with its scenarios and their respective traces of execution.

Figure 1 illustrates the relationship between features, scenarios, and methods, where the features are on the top in blue color, the scenarios are in the middle in yellow color and the methods are on the bottom of the Figure in green color. We should note that, from this structure, the upper levels of the OP regarding customer, user and system mode may be also complemented if the set of *User-Defined Tags* are specified and then previously analysed by ARM.

In order to conclude the steps of OP generation, the probability of occurrence of each mapped entity must be performed. As in traditional OP generation approaches, these probabilities must be obtained from the domain context. Such information can be obtained from interaction with domain experts or experienced users [19]. Thus, high-level entities

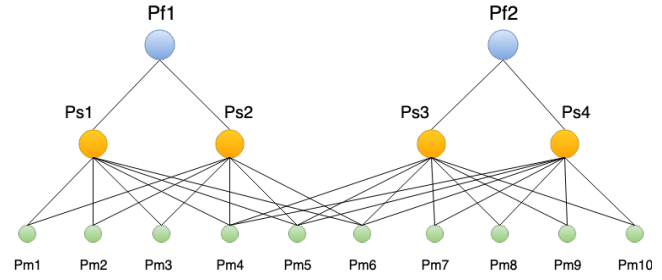


Figure 1: Map of Features, Scenarios and Methods.

can receive estimated probability information from the highest OP level to the level of **key input variables**. With this information, the software execution trace enables the distribution of probabilities between the levels of **Functional Profile** and **Operational Profile**, completing the OP.

Other relevant information is the number of *Impacted Features* (IF) by a given code entity such as methods or classes. This information is straightforward since the entities have already been mapped.

In this way, the Analyzer Module provides the information obtained and the analysis performed to the Visualizer Module so that those responsible for the testing process adopt the strategy they want to distribute the testing effort and to prioritize the test cases.

The Visualizer Module

The Visualizer module enables: (1) the global view of the SUT, making evident its traceability; (2) performing analyzes from the data crossing and (3) using the generated data to support the test case prioritization according to the desired strategy. This Module is responsible for presenting the outcome from the Analyzer Module. Among the information presented, we highlight the groups of entities based on criteria such as complexity, *Program Spectrum* and OP.

The global view of the software automates the process of identifying the requirements in the source code, which is recognized as an essential activity for the software maintenance and evolution process [6].

This set of visualizations and information of the software entities enables the application of several techniques for prioritizing test cases, from black-box techniques to white-box techniques.

4 CASE STUDY

We evaluate our Feature-Trace approach to its ability to analyze different metrics related to the software testing process, and thereby enable the distribution of the test effort throughout the source code.

We carried out a case study in **Diaspora**: a social network with 12,226 *stars* and 2,915 *forks* in its repository in *GitHub*². It is implemented in Ruby and has a total of 72,674 lines of code, out of which 40,546 lines of Ruby code with 1,571 methods, and a total of 2,975 lines of *Gherkin* code, distributed throughout 68 BDD features. It also has a set of 2679 unit and integration test cases.

The initial process of the analysis occurs with the static analysis of the SUT source code. This analysis is done in an automated way by the Feature-Trace tool. Some information is obtained from the use of the *Excellent*³ tool, which makes it possible to obtain the Cyclomatic Complexity, ABC Score and Number of lines of the method. The execution of the BDD scenarios is done with the support of the *Cucumber*⁴ tool, being executed for each scenario present and also integrated into our Feature-Trace tool. In order to obtain the execution trace of the scenarios and unit/integration tests, we use the *Simplecov*⁵ tool.

Results and Analysis

Table 1 shows that we obtain the OP, given that the highest levels of OP were obtained, the number of IF (impacted features) and the number of test cases that exercise each SUT method. We should note that, although Feature-Trace also gathers static information regarding the software complexity, such as Cyclomatic Complexity, ABC Score and Number of Lines, this information has been removed from Table 1 to highlight those information at stake as an outcome of our approach. In this way, the team should use this information to weight and distribute the testing effort throughout the source code.

Table 1: Methods information obtained with Feature-Trace.

Method	Occ. Prob.	Imp. Feat.	Spec.
<i>person_image_link</i> <i>PeopleHelper</i> (M1)	88.4785%	61	2679
<i>current_user_person_contact</i> <i>PersonPresenter</i> (M2)	53.8023%	45	92
<i>3. extension_whitelist</i> <i>UnprocessedImage</i> (M3)	16.3767%	14	116

Regarding the global view of the SUT, Figure 2 shows the representation of the “*Preview Posts in the Stream*” feature (in orange), its scenarios (in purple), and corresponding invoked methods (in green). In a top-down view, the image allows a broad view of the set of methods that are involved with all scenarios (automatically positioned at the center of the

graph). Therefore, these are the essential methods for running the Feature as a whole. It is also important to highlight the specific methods of each feature usage scenario (prominently positioned). This graph-view perspective represents the mapping of entities of the Diaspora feature “*Preview Posts in the Stream*” and makes evident the software traceability.

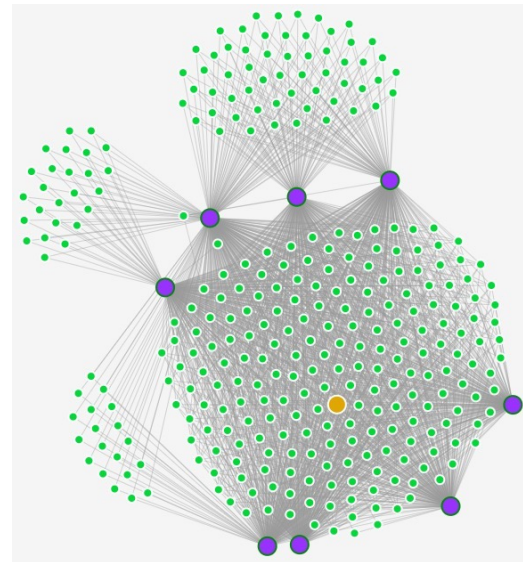


Figure 2: Global view of the feature “*preview posts in the stream*”.

5 RELATED WORK

Several contributions in the literature have proposed means to enable the obtaining of the OP, while minimizing the effort for its application. A widely used strategy is the analysis of system usage logs from the production environment [13, 22, 29].

Among those contributions that aim to approximate the software requirements to the test artifacts, we highlight the work proposed by Per Runeson et al. [27], which suggests the relationship between the Use-Case Model and OP. Unlike our Feature-Trace approach, the execution of those requirements artifacts based on Use-Case Models is performed manually, as opposed to using the BDD approach.

In addition, some contributions focus on the process of prioritizing test cases. Miranda et al. [3] propose *covrel*, where the concepts of OP (black-box information) and *Program Spectrum* (white-box information) are used to maximize the growth of SUT reliability throughout the testing process.

About the variety of techniques and strategies of prioritization of test cases that can be used during a process of prioritization of test cases, Ouriques et al. [25] make a survey of some techniques related to the context of model-based testing (MTB).

²<https://github.com/diaspora/diaspora>

³<https://github.com/simplabs/excellent>

⁴<https://cucumber.io/>

⁵<https://github.com/colszowka/simplecov>

The use of test tools made available as Web Services was discussed by [10] when proposing the tool JaBUTiService, presenting some benefits related to this context: Ease of use, Availability, Version Control, Integration, Orchestration, and Choreography of software engineering services and Comparison. Due to the architectural character of the tool proposed in this work, we have added some of these advantages, which deserve future evaluation.

6 CONCLUSION

The approach presented in this paper, Feature-Trace, is based on the software traceability, derived from the use of the BDD approach, to extract relevant information that allows the application of several strategies and techniques of prioritization of test cases, including the concept of OP. According to the presented case study, using the Diaspora social network, it was possible to notice that the development team involved can aggregate, without much effort, relevant information to guide the process of software maintenance, evolution, and testing. As future work, we plan the evaluation of the Feature-Trace approach in conjunction with some test case prioritization approaches, such as *Covrel* and *FAST*, aiming, in particular, to analyze the effort expended during the prioritization.

REFERENCES

- [1] Fabian Beck, Bogdan Dit, Jaleo Velasco-Madden, Daniel Weiskopf, and Denys Poshyvanyk. 2015. Rethinking user interfaces for feature location. In *Proceedings of the 2015 IEEE 23rd Intl. Conf. on Program Comprehension*. IEEE Press, 151–162.
- [2] Kent Beck. 2003. *Test-driven development: by example*. Addison-Wesley Professional.
- [3] Antonia Bertolino, Breno Miranda, Roberto Pietrantuono, and Stefano Russo. 2017. Adaptive coverage and operational profile-based testing for reliability improvement. In *Proceedings of the 39th International Conference on Software Engineering*. IEEE Press, 541–551.
- [4] Elizabeth Bjarnason, Per Runeson, Markus Borg, Michael Unterkalmsteiner, Emelie Engström, Björn Regnell, Giedre Sabaliauskaite, Annabella Loconsole, Tony Gorscheck, and Robert Feldt. 2014. Challenges and practices in aligning requirements with verification and validation: a case study of six companies. *Empirical Software Engineering* 19, 6 (2014), 1809–1855.
- [5] Cagatay Catal and Deepti Mishra. 2013. Test case prioritization: a systematic mapping study. *Software Quality Journal* 21, 3 (2013), 445–478.
- [6] Bogdan Dit, Meghan Revelle, Malcom Gethers, and Denys Poshyvanyk. 2013. Feature location in source code: a taxonomy and survey. *Journal of software: Evolution and Process* 25, 1 (2013), 53–95.
- [7] Marc Eaddy, Alfred V Aho, Giuliano Antoniol, and Yann-Gaël Guéhéneuc. 2008. Cerberus: Tracing requirements to source code using information retrieval, dynamic analysis, and program analysis. In *2008 16th IEEE Intl. Conf. on Program Comprehension*. Ieee, 53–62.
- [8] Christof Ebert and James Cain. 2016. Cyclomatic complexity. *IEEE software* 33, 6 (2016), 27–29.
- [9] Thomas Eisenbarth, Rainer Koschke, and Daniel Simon. 2003. Locating features in source code. *IEEE Transactions on software engineering* 29, 3 (2003), 210–224.
- [10] Marcelo Medeiros Eler, Andre Takeshi Endo, Paulo Cesar Masiero, Marcio Eduardo Delamaro, Jose Carlos Maldonado, Auri Marcelo Rizzo Vincenzi, Marcos Lordello Chaim, and Delano Medeiros Beder. 2009. JaBUTiService: a web service for structural testing of java programs. In *2009 33rd Annual IEEE Software Engineering Workshop*. IEEE, 69–76.
- [11] Eric Evans. 2004. *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley Professional.
- [12] Jerry Fitzpatrick. 1997. Applying the ABC metric to C, C++, and Java. *More C++ gems* (1997), 245–264.
- [13] Ahmed E Hassan, Daryl J Martin, Parminder Flora, Paul Mansfield, and Dave Dietz. 2008. An industrial case study of customizing operational profiles using log compression. In *Proceedings of the 30th international conference on Software engineering*. ACM, 713–723.
- [14] Christopher Henard, Mike Papadakis, Mark Harman, Yue Jia, and Yves Le Traon. 2016. Comparing white-box and black-box test prioritization. In *Software Engineering (ICSE), 2016 IEEE/ACM 38th International Conference on*. IEEE, 523–534.
- [15] Yi Li, Chenguang Zhu, Julia Rubin, and Marsha Chechik. 2017. FHistory: Locating features in version histories. In *Proceedings of the 21st International Systems and Software Product Line Conference-Volume A*. ACM, 49–58.
- [16] Dapeng Liu, Andrian Marcus, Denys Poshyvanyk, and Vaclav Rajlich. 2007. Feature location via information retrieval based filtering of a single scenario execution trace. In *Proceedings of the twenty-second IEEE/ACM international conference on Automated software engineering*. ACM, 234–243.
- [17] Thomas J McCabe. 1976. A complexity measure. *IEEE Transactions on software Engineering* 4 (1976), 308–320.
- [18] Breno Miranda, Emilio Cruciani, Roberto Verdecchia, and Antonia Bertolino. 2018. Fast approaches to scalable similarity-based test case prioritization. In *Proceedings of the 40th International Conference on Software Engineering*. ACM, 222–232.
- [19] John D Musa. 1993. Operational profiles in software-reliability engineering. *IEEE software* 2 (1993), 14–32.
- [20] John D Musa. 1996. The operational profile. In *Reliability and Maintenance of Complex Systems*. Springer, 333–344.
- [21] Glenford J Myers, Corey Sandler, and Tom Badgett. 2011. *The art of software testing*. John Wiley & Sons, New Jersey, USA.
- [22] Meiyappan Nagappan, Kesheng Wu, and Mladen A Vouk. 2009. Efficiently extracting operational profiles from execution logs using suffix arrays. In *Software Reliability Engineering, 2009. ISSRE'09. 20th International Symposium on*. IEEE, 41–50.
- [23] Tanzeem Bin Noor and Hadi Hemmati. 2015. A similarity-based approach for test case prioritization using historical failure data. In *2015 IEEE 26th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 58–68.
- [24] Dan North. 2003. Introducing BDD. <https://dannorth.net/introducing-bdd/>
- [25] Joao Felipe S Ouriques, Emanuela G Cartaxo, and Patrícia DL Machado. 2018. Test case prioritization techniques for model-based testing: a replicated study. *Software Quality Journal* (2018), 1–32.
- [26] Patrick Rempel and Parick Mäder. 2017. Preventing defects: The impact of requirements traceability completeness on software quality. *IEEE Transactions on Software Engineering* 43, 8 (2017), 777–797.
- [27] Per Runeson and Björn Regnell. 1998. Derivation of an integrated operational profile and use case model. In *Software Reliability Engineering, 1998. Proceedings. The Ninth International Symposium on*. IEEE, 70–79.
- [28] John Ferguson Smart. 2014. *BDD in Action*. Manning Publications.
- [29] Hany EL Yamany and Miriam AM Capretz. 2007. A Multi-Agent Framework for Building an Automatic Operational Profile. In *Advances and Innovations in Systems, Computing Sciences and Software Engineering*. Springer, 161–166.